

Chapter: LeetCode Notes (Question 803)

Authored by: Arpit Raj, LNMIIT Jaipur

Date: August 4, 2026

© Copyright 2026 Arpit Raj. All rights reserved.

You are given an $m \times n$ binary grid, where each 1 represents a brick and 0 represents an empty space. A brick is stable if:

- It is directly connected to the top of the grid, or
- At least one other brick in its four adjacent cells is stable.

hits, which is a sequence of erasures we want to apply. Each time we want to erase the brick at the location $hits[i] = (row_i, col_i)$. The brick on that location (if it exists) will disappear. Some other bricks may no longer be stable because of that erasure and will fall. Once a brick falls, it is immediately erased from the grid (i.e., it does not land on other stable bricks).

Return an array `result`, where each `result[i]` is the number of bricks that will fall after the i th erasure is applied.

Step 0 : Why does the normal DSU fail?

Suppose

1	1	1
1	1	1

Now hit $(0, 1)$

Grid becomes

1	0	1
1	1	1

Now several bricks may fall, we cant delete component in dsu , because after all merge it forgets how it was built

So

Delete 

Add 

Thinking Process

Instead of thinking

Original



Hit 1



Hit 2



Hit 3



think

Remove everything first



Add back Hit3



Add back Hit2



Add back Hit1

Now we're only Adding

Original

1	1
1	1

Hits: $(0, 1)$, $(1, 1)$

First remove all hits

Grid

1	0
1	0

Now

Add



(1, 1)



Add



(0, 1)

Every operation is Addition

Step 3 : Virtual Roof ★★★★★

Instead of checking

Can this brick reach top?

every time,
create one extra node.
Suppose

$\text{roof} = n * m$

DSU size becomes
 $n * m + 1$

Example

1	1	0
1	0	1

→ Numbering →

0	1	2
3	4	5

Roof = 6

before adding
roof component

After adding
roof component

has
5 bricks

has
9 bricks

Difference

$$9 - 5 = 4$$

What happened?

Exactly **4** new bricks became stable.

But one of them is the brick we ourselves added.

Question asks

How many OTHER bricks?

Hence answer = after-before-1;

Complete Algorithm

- **Step 1:** Copy grid.
- **Step 2:** Remove every hit.

Original ↓ Grid after all removals

- **Step 3:** Build DSU on this final grid.
Connect adjacent bricks.
Connect first row to Roof.
- **Step 4:** Process hits backwards.
For every hit:

Before roof size
↓
Add brick
↓
Union neighbours
↓
After roof size
↓
after-before-1

Store answer.

- **Step 5:** Reverse answers.

Done.

Important Details

Order of Operations

Suppose we're adding back a brick.

Should we

Union with neighbours first ↓ Connect to Roof

or

Connect to Roof first ↓ Union neighbours

Actually, either order works.

I personally prefer

Connect to Roof first (if first row)



Union neighbours

because it follows the logic:

"This brick has appeared. If it's on the roof, attach it immediately. Then attach its neighbours."

What if the brick was never there?

Suppose hit is (2,3) but originally `grid[2][3]==0`

Meaning there was never a brick there.

Example

Original

1	0
1	1

Hit (0,1)

Nothing happens.

While reversing, should we add a brick?

No.

Because there was never a brick.

So before removing all hits, we must remember **original grid**

```
graph TD; A[before = roof size] --> B[Restore brick]; B --> C[Union roof if first row]; C --> D[Union neighbours]; D --> E[after = roof size]; E --> F[ans = max(0, after-before-1)];
```

C++ Solution Code

```
class dsu{
public:

    vector<int> parent;
    vector<int> size;

    dsu(int n){

        parent.resize(n);
        size.resize(n,1);

        for(int i=0;i<n;i++){
            parent[i]=i;
        }
    }

    int findpar(int node){

        if(parent[node]==node){
            return node;
        }

        return parent[node]=findpar(parent[node]);
    }

    void unionit(int u,int v){

        int pu=findpar(u);
        int pv=findpar(v);

        if(pu==pv){
            return;
        }

        if(size[pu]<size[pv]){
            swap(pu,pv);
        }

        parent[pv]=pu;
        size[pu]+=size[pv];
    }
};

class Solution {
public:
    vector<int> hitBricks(vector<vector<int>>& grid, vector<vector<int>>& hits) {

        int n=grid.size();
        int m=grid[0].size();
```

```

vector<vector<int>> temp=grid;
//never modify the original array
//kyunki jab reverse me brick add krenge to pata hona chahiye
//ki yaha pe original array me brick tha bhi ya nahi

//eg maan le ek cell me brick tha hi nhi
//waha se ham brick hataye bhi to kuch nhi hoga

//hence jab reverse process krenge
//to waha pe brick add ho jaayega even though original me nhi tha
//isliye if orig[r][c]==0 continue;

int roof=n*m;
//1 extra node ham khud se add krenge , virtual boundary roof
//0 se n*m-1 waale grid nodes he

dsu ds((n*m)+1);
vector<pair<int,int>> dir={{-1,0},{0,-1},{1,0},{0,1}};

//now to begin our rev processing ek baar final state tak phuchna padega
for(auto &it:hits){

    int r=it[0];
    int c=it[1];

    if(temp[r][c]==1){
        temp[r][c]=0;
    }

}

//ab start krenge
for(int r=0;r<n;r++){
    for(int c=0;c<m;c++){

        if(temp[r][c]==0) continue;

        int curr=r*m+c;

        if(r==0){
            ds.unionit(curr,roof);
        }

        for(auto &it:dir){
            int nr=r+it.first;
            int nc=c+it.second;

            int next=nr*m+nc;
            if(nr>=0 && nc>=0 && nr<n && nc<m){
                if(temp[nr][nc]==0) continue;
                ds.unionit(next,curr);
            }
        }
    }
}

```

```

    }
}

//ab finally reversing chaalu krenge now that we have final grid

vector<int> ans;
for(int i=hits.size()-1 ;i>=0;i--){

    int r=hits[i][0];
    int c=hits[i][1];

    if(grid[r][c]==0){ //agar original grid me brick tha hi nhi
                        //to iss brick ne kuch giraya hi nahi kabhi
                        //hence no change
        ans.push_back(0);
        continue;
    }

    int before=ds.size[ds.findpar(roof)];
    temp[r][c]=1; //restoring that brick

    int curr=r*m+c;
    if(r==0){
        ds.unionit(curr,roof);
    }

    //iss brick ke aane ke baad aur bhi stable honge

    for(auto &it:dir){
        int nr=r+it.first;
        int nc=c+it.second;
        int next=nr*m+nc;

        if(nr>=0 && nc>=0 && nr<n && nc<m){
            if(temp[nr][nc]==0) continue;
            ds.unionit(curr,next);
        }
    }

    int after=ds.size[ds.findpar(roof)];
    ans.push_back(max(0,after-before-1));

    //-1 kyunki ek roof element hamne khud add kiya tha
}

reverse(ans.begin(),ans.end());
return ans;

}
};

```